

整体设计：两产品生态与界面划分

本文档为设计文档体系的顶层文件，定义 BitFS 与 Metanet 两产品的职责边界、三层架构和共享设计原则。

设计文档体系： - 整体设计（本文档） — 两产品生态、三层架构、界面划分 - [BitFS 设计](#) — 去中心化加密文件系统协议 - [Metanet 设计](#) — 去中心化 CDN 网络

一、两产品生态概览

维度	BitFS	Metanet
定位	去中心化加密文件系统协议	去中心化 CDN 网络
域名	bitfs.org	metanet.org
CLI	<code>bitfs</code> (文件所有者/访问者)	<code>metanet</code> (CDN 节点运营商)
用户	终端用户、AI Agent	内容所有者、节点运营商
货币	BSV	MNT Token
类比	IPFS (协议层)	Filecoin (激励层)

关键区别：Filecoin 激励存储 (Proof-of-Replication)；Metanet 激励检索 (x402 按次付费)。热门内容自组织复制，冷数据自然淘汰——市场驱动而非强制冗余。

二、三层架构

Layer 1: BSV 主链 文件所有权 · Metanet DAG · HTLC 原子交换 · x402 付费	货币: BSV
Layer 2: 自托管 Daemon bitfs daemon (LFCP) · 内容存储 · 直接服务	无需代币

Layer 3: Metanet Chain

货币: MNT

去中心化 CDN · 存储合约 · 支付通道 · 挖矿奖励

层级	职责	面向	支付方式
Layer 1 — BSV 主链	文件元数据、所有权证明、HTLC 交易、x402 检索费	终端用户、AI Agent	BSV
Layer 2 — 自托管 Daemon	本地内容存储、LFCP 直接服务、无第三方依赖	文件所有者 (自服务)	无
Layer 3 — Metanet Chain	CDN 托管、节点激励、支付通道批量结算	内容所有者、节点运营商	MNT

数据无需跨链: BSV 主链只存元数据 (Metanet DAG 交易); 内容数据始终在链下流转 (Daemon 或 CDN 节点)。Metanet Chain 管理经济激励, 不承载文件内容。

三、界面划分

3.1 共享核心库 (libbitfs)

两产品共享同一 Go 库, 避免重复实现:

```
libbitfs/
├── method42/      # Method 42 ECDH 加密 (secp256k1 + AES-256-GCM)
├── metanet/       # Metanet DAG 解析 (inode, dirent, 软/硬链接)
├── spv/           # SPV 轻节点 (本地 tx + Merkle proof)
├── storage/       # 内容存储抽象 (链下/链上)
├── paymail/       # Paymail 身份解析
├── tx/            # BSV 交易构造 (go-sdk)
├── x402/          # x402 支付协议 + Token 预购
└── revshare/     # Revenue Share / ISO 证券化
```

3.2 独立二进制

二进制	面向	核心功能
bitfs	文件所有者、访问者、AI Agent	put/get/ls/cat/rm/mv/cp、sell、wallet、daemon
metanet	CDN 节点运营商	init/start/stop、status、contracts、peers、mine

bitfs 用户无需安装 metanet；metanet 节点内嵌 libbitfs 以解密和服务内容。

3.3 链间职责

终端用户 / Agent	→	BSV 主链	→	文件元数据 + x402/HTLC 支付
↓ 内容哈希引用 (非跨链)				
内容所有者	→	Metanet Chain	→	CDN 托管合约 + MNT 结算
		Metanet Node	→	实际内容存储与检索服务

四、双币种模型

币种	使用场景	接触者
BSV	x402 检索费、HTLC 文件购买、Metanet DAG 交易手续费	所有用户
MNT	CDN 托管费、挖矿奖励、节点间批发结算	仅内容所有者和节点运营商

设计目标：普通用户只接触 BSV。MNT 是运营商侧的内部结算代币，对终端用户完全透明。

五、设计文档体系

design/
├─ 0-OverallDesign.zh.md

← 本文档：整体设计，界面划分

— bitfs/	← BitFS 文件系统协议设计
— 1-ConceptDesign.zh.md	概念设计（愿景，架构，设计决策）
— 2-SystemDesign.zh.md	系统设计（模块，接口，数据流）
— 3-DetailedDesign.zh.md	详细设计（算法，数据结构，协议）
— 4-TestDesign.zh.md	测试设计（~938 测试用例）
— metanet/	← Metanet CDN 网络设计
— 1-ConceptDesign.zh.md	概念设计（CDN 模型，经济设计）
— 2-SystemDesign.zh.md	系统设计（节点，合约，支付通道）
— 3-DetailedDesign.zh.md	详细设计（共识，挖矿，结算）
— 4-TestDesign.zh.md	测试设计

每个产品的设计文档遵循四层结构：概念设计 → 系统设计 → 详细设计 → 测试设计。

六、共享设计原则

1. Unix 哲学 — 每个工具做一件事，可管道组合（`bcats txid | jq .`）
2. Agent-first — CLI 输出结构化 (JSON)，x402 付费墙即可编程支付接口
3. 默认加密 — Method 42 (ECDH + BIP32)，所有数据加密存储，密钥由文件路径确定性派生
4. SPV 模式 — 本地保存交易 + Merkle proof，从不查询区块链全节点
5. BSV 同构 — Metanet Chain 使用与 BSV 相同的交易格式和 Script 引擎
6. BRC 标准兼容 — 遵循 BSV Association 的 BRC 标准体系
7. BSV 官方库 — 唯一 BSV 依赖为 `github.com/bsv-blockchain/go-sdk`
8. 元数据与内容分离 — 链上只存元数据 (Metanet DAG)，内容独立存储